Importante
Puedo eliminar la publi de este documento con 1 coin
¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins



Inteligencia Artificial

Tema 7 — Aprendizaje

Resumen estructurado para examen

Basado en las diapositivas del Dpto. de Informática · UJA

Ingeniería Informática · Universidad de Jaén

Curso 2025/2026

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Índice

1. Introducción: ¿Qué es el Aprendizaje? 3
1.1. Definición . . . 3
1.2. Agente de Aprendizaje . . . 3
1.3. Aprendizaje Automático (Machine Learning) . . . 3
1.3.1. Tipos de Aprendizaje Automático . . . 3

2. Conceptos Clave del Aprendizaje 4
2.1. Conjuntos de Datos . . . 4
2.2. Clasificación vs. Regresión . . . 4
2.3. Parámetros vs. Hiperparámetros . . . 5
2.4. Aprendizaje Basado en Modelos vs. Basado en Instancias . . . 5
2.5. Aprendizaje Superficial vs. Profundo . . . 5

3. Aprendizaje Inductivo Supervisado 5
3.1. Definición Formal . . . 5
3.2. Selección de Hipótesis: Navaja de Ockham . . . 5
3.3. Evaluación del Rendimiento . . . 6
3.3.1. Métricas en Clasificación . . . 6
3.3.2. Métricas en Regresión . . . 6
3.3.3. Sobreajuste (*Overfitting*) . . . 6

4. Árboles de Decisión 6
4.1. Definición y Estructura . . . 6
4.2. Algoritmo de Aprendizaje . . . 7
4.3. Expresividad y Limitaciones . . . 8

5. SVM — Máquina de Vectores Soporte 8
5.1. Idea Central . . . 8
5.2. Vectores Soporte y Margen . . . 8

6. Aprendizaje Basado en Instancias: kNN 8
6.1. Características . . . 8
6.2. Funcionamiento . . . 9
6.3. Hiperparámetro k y su Impacto . . . 9

7. Redes Neuronales Artificiales (RNA) 10
7.1. Definición y Modelo de Neurona . . . 10
7.2. Arquitectura . . . 10
7.3. Entrenamiento: Backpropagation . . . 10
7.4. Tipos de RNA . . . 11

8. Aprendizaje Profundo (*Deep Learning*) 12
8.1. Definición y Motivación . . . 12
8.2. Funcionamiento Jerárquico . . . 12
8.3. Reto: *Vanishing Gradient* . . . 12
8.4. Principales Arquitecturas DL . . . 12
8.4.1. Autoencoders . . . 12

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Elimina la publicidad de este documento con 1 coin

8.4.2. Redes Convolucionales (CNN) 12
8.4.3. Redes Recurrentes (RNN / LSTM) 13
8.4.4. Otras Arquitecturas 13
8.5. ¿Por Qué Tiene Tanto Éxito el DL Actualmente? 13

9. Resumen Comparativo de Técnicas **14**

10.Puntos Clave para el Examen **14**

Bibliografía **16**

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Elimina la publicidad de este documento con 1 coin

Importante
Puedo eliminar la publi de este documento con 1 coin
¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

1. Introducción: ¿Qué es el Aprendizaje?

1.1. Definición

El **aprendizaje** en IA consiste en utilizar las percepciones para mejorar la habilidad del agente de actuar en el futuro, sin haber sido programado explícitamente para la tarea.

Algunas definiciones relevantes en el ámbito de la IA:

- **H. Simon:** cualquier cambio en un sistema que permita realizar la misma tarea de manera más eficiente.
- **T.M. Mitchell:** se dice que aprendemos de la experiencia al realizar alguna tarea si la realización mejora con la experiencia respecto a alguna medida de rendimiento.

1.2. Agente de Aprendizaje

Un agente de aprendizaje tiene dos elementos principales:

- **Elemento de acción (Performance element):** decide qué acciones llevar a cabo.
- **Elemento de aprendizaje (Learning element):** modifica el elemento de acción para tomar mejores decisiones en el futuro.

El diseño del elemento de aprendizaje depende de:

1. Qué componentes del elemento de acción deben aprenderse.
2. Qué realimentación (*feedback*) está disponible.
3. Qué tipo de medida de rendimiento se utiliza.
4. Qué representación se usa para los componentes.

1.3. Aprendizaje Automático (Machine Learning)

Objetivo: Construir algoritmos que, a partir de datos y percepciones, mejoran su rendimiento en una tarea sin haber sido programados explícitamente para ella.

1.3.1. Tipos de Aprendizaje Automático

- **Supervisado:** aprende un modelo a partir de ejemplos *etiquetados* (entrada + salida correcta). Es el más común. *Ej: clasificación de imágenes, detección de spam.*
- **No supervisado:** aprende patrones a partir de entradas sin salidas especificadas. *Ej: clustering, reducción de dimensionalidad.*
- **Semi-supervisado:** combina datos etiquetados y no etiquetados.
- **Por refuerzo:** el agente aprende mediante recompensas o penalizaciones según las acciones ejecutadas en un entorno. La decisión es secuencial con objetivo a medio/largo plazo. *Ej: juegos, robótica.*

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Agente de aprendizaje

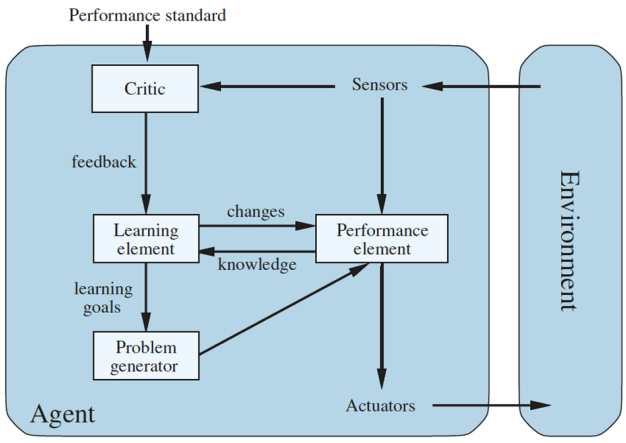


Figura 1: Arquitectura de un agente de aprendizaje: el crítico evalúa el rendimiento y el elemento de aprendizaje modifica la actuación. (Fuente: Diapositiva 6 del temario)

2. Conceptos Clave del Aprendizaje

2.1. Conjuntos de Datos

- **Conjunto de entrenamiento:** datos usados para aprender el modelo/función. Puede subdividirse en entrenamiento + validación para ajuste de hiperparámetros.
- **Conjunto de test:** datos usados para estimar el rendimiento del modelo con datos nuevos (nunca vistos durante el entrenamiento).

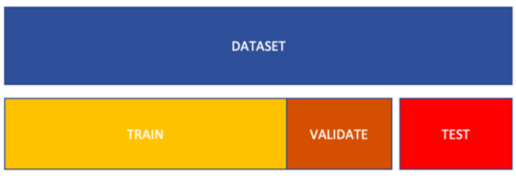


Figura 2: División del *dataset* en Train / Validate / Test. (Fuente: Diapositiva 12 del temario)

2.2. Clasificación vs. Regresión

Tipo	Salida	Ejemplo
Clasificación	Valor discreto (etiqueta)	Detección de spam
Regresión	Valor numérico continuo	Precio de un inmueble

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH
Elimina la publicidad de este documento con 1 coin

2.3. Parámetros vs. Hiperparámetros

- **Hiperparámetros:** propiedades del algoritmo de aprendizaje que *no* aprende el algoritmo; deben configurarlos el experto antes del entrenamiento. *Ej: el valor de k en kNN.*
- **Parámetros:** variables que definen el modelo aprendido, determinadas por el algoritmo a partir del conjunto de entrenamiento. *Ej: pesos en una red neuronal.*

2.4. Aprendizaje Basado en Modelos vs. Basado en Instancias

- **Basado en modelos:** el entrenamiento produce un modelo compacto; luego se descarta el conjunto de entrenamiento. *Ej: árbol de decisión C4.5.*
- **Basado en instancias:** el conjunto de entrenamiento es el modelo; se consulta en cada predicción. *Ej: kNN.*

2.5. Aprendizaje Superficial vs. Profundo

- **Superficial (*shallow*):** los parámetros se aprenden directamente de las características del dato. *Ej: árbol de decisión.*
- **Profundo (*deep*):** la mayoría de parámetros se aprenden a partir de la salida de capas anteriores, no de los datos originales. *Ej: CNN, Transformers.*

3. Aprendizaje Inductivo Supervisado

3.1. Definición Formal

> Dado un conjunto de ejemplos $\{(\mathbf{x}, f(\mathbf{x}))\}$, el objetivo es encontrar una hipótesis h tal que $h \approx f$ para nuevos ejemplos no vistos.

- f: función objetivo (desconocida).
- h: hipótesis aprendida.
- El **espacio de hipótesis** $\mathcal{H}$ es el conjunto de funciones candidatas consideradas.
- Una hipótesis es **consistente** si predice correctamente todos los ejemplos de entrenamiento.
- Una hipótesis **generaliza bien** si predice correctamente ejemplos nuevos (conjunto de test).

3.2. Selección de Hipótesis: Navaja de Ockham

> Ante múltiples hipótesis consistentes con los datos, se prefiere la **más sencilla**. Esto es el **principio de la navaja de Ockham** y es la base teórica de la regularización en ML.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Elimina la publicidad de este documento con 1 coin

Importante
Puedo eliminar la publi de este documento con 1 coin
¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

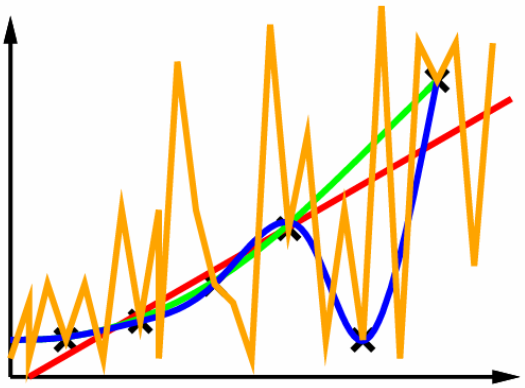


Figura 3: Múltiples hipótesis consistentes con el mismo conjunto de entrenamiento; se prefiere la más simple. (Fuente: Diapositiva 19 del temario)

3.3. Evaluación del Rendimiento

Para comprobar que $h \approx f$ se prueba h sobre el **conjunto de test**.

3.3.1. Métricas en Clasificación

Matriz de confusión, exactitud (*accuracy*), precisión/sensibilidad (*precision/recall*), área bajo la curva ROC, etc.

3.3.2. Métricas en Regresión

Error cuadrático medio (MSE), calculado por separado en entrenamiento y test.

3.3.3. Sobreajuste (*Overfitting*)

> Si la medida de rendimiento en entrenamiento es **sustancialmente mayor** que en test, hay **sobreajuste**: el modelo memoriza los datos de entrenamiento y pierde capacidad de generalización.

4. Árboles de Decisión

4.1. Definición y Estructura

> Un **árbol de decisión** es un modelo basado en modelos que clasifica (o regresa) una instancia recorriendo una estructura jerárquica de condiciones sobre sus atributos.

- **Nodo de decisión:** especifica una prueba simple sobre un atributo.
- **Rama:** posible resultado de la prueba.
- **Hoja:** categoría de salida (clasificación) o valor continuo (regresión).

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

- El árbol es directamente expresable como un conjunto de reglas `SI-ENTONCES`.

Problema 1: Clasificación de plantas *iris* (*iris setosa*, *iris versicolor* o *iris virginica*) en función de las características del ancho y largo de su pétalo y/o sépalo.

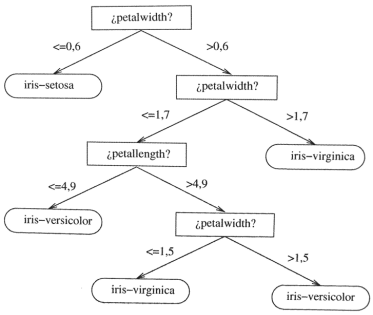


Figura 4: Árbol de decisión para clasificación de plantas *iris* según el ancho y largo del pétalo/sépalo. (Fuente: Diapositiva 27 del temario)

4.2. Algoritmo de Aprendizaje

Estrategia **divide y vencerás** top-down:

1. Partir de todos los ejemplos en la raíz.
2. Seleccionar el atributo más significativo según una medida de **ganancia de información** (criterio de selección).
3. Distribuir los ejemplos entre los descendientes según los valores del atributo.
4. Repetir recursivamente hasta que todos los ejemplos de un nodo pertenezcan a la misma clase, no queden atributos o los ejemplos sean vacíos.

function LEARN-DECISION-TREE($examples, attributes, parent_examples$) **returns** a tree

if $examples$ is empty **then return** PLURALITY-VALUE($parent_examples$)
else if all $examples$ have the same classification **then return** the classification
else if $attributes$ is empty **then return** PLURALITY-VALUE($examples$)
else
  $A \leftarrow \text{argmax}_{a \in attributes}$ IMPORTANCE($a, examples$)
  $tree \leftarrow$ a new decision tree with root test A
  for each value v of A **do**
    $exs \leftarrow \{e : e \in examples$ **and** $e.A = v\}$
    $subtree \leftarrow$ LEARN-DECISION-TREE($exs, attributes - A, examples$)
    add a branch to $tree$ with label $(A = v)$ and subtree $subtree$
  return $tree$

Figura 5: Pseudocódigo del algoritmo `Learn-Decision-Tree`. (Fuente: Diapositiva 30 del temario)

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH
Elimina la publicidad de este documento con 1 coin

4.3. Expresividad y Limitaciones

- Pueden expresar *cualquier* función booleana de los atributos de entrada.
- Un árbol muy profundo memorizará el entrenamiento (*overfitting*); se prefieren árboles **más compactos** (navaja de Ockham).
- Algoritmos clásicos: **ID3**, **C4.5**, **CART**.

5. SVM — Máquina de Vectores Soporte

5.1. Idea Central

Las **SVM** (*Support Vector Machines*) buscan el **hiperplano de máximo margen** que separa las clases en el espacio de características.

- Se basan en el principio de **minimización del riesgo estructural**: prefieren modelos fiables con buen margen antes que modelos que cometen pocos errores en entrenamiento pero generalizan mal.
- Cada instancia de N variables se trata como un punto en $\mathbb{R}^N$.
- El hiperplano separador tiene ecuación $\mathbf{w} \cdot \mathbf{x} - b = 0$, con $\mathbf{w}$ vector de pesos.
- El objetivo de aprendizaje es encontrar $\mathbf{w}$ y b óptimos (problema de optimización cuadrática).

5.2. Vectores Soporte y Margen

- Los **vectores soporte** son los ejemplos más cercanos al hiperplano separador (uno por cada clase).
- El **margen** es la distancia entre los dos hiperplanos de margen ($\mathbf{w} \cdot \mathbf{x} - b = \pm 1$). La SVM lo maximiza.
- Un margen mayor implica mejor generalización.

Para datos **no linealmente separables** se usa el **truco del kernel**: proyectar los datos a un espacio de mayor dimensión donde sí sean separables, sin calcular explícitamente dicha proyección.

6. Aprendizaje Basado en Instancias: kNN

6.1. Características

El algoritmo **k-Nearest Neighbours (kNN)** es un método de aprendizaje basado en instancias y no paramétrico: usa todo el conjunto de entrenamiento como modelo.

Diferencia clave con métodos basados en modelos: kNN no extrae ningún modelo compacto; los ejemplos de entrenamiento se conservan y se consultan en cada predicción.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Elimina la publicidad de este documento con 1 coin

Importante
Puedo eliminar la publi de este documento con 1 coin
¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

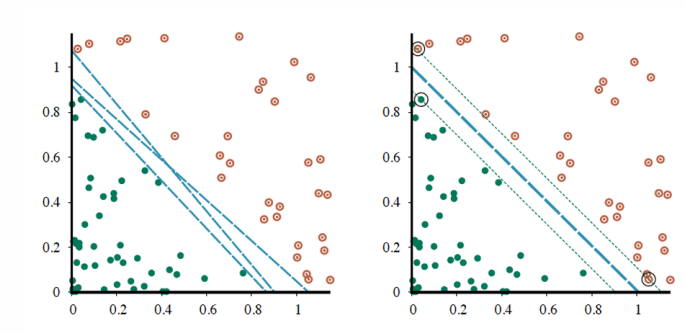


Figura 6: Separadores lineales candidatos (izq.) y separador de máximo margen con vectores soporte marcados (der.). (Fuente: Diapositiva 34 del temario)

6.2. Funcionamiento

Dado un nuevo ejemplo $\mathbf{x}$:

1. Calcular la distancia de $\mathbf{x}$ a todos los ejemplos del conjunto de entrenamiento (euclidiana, Hamming, Mahalanobis...).
2. Seleccionar los k ejemplos más cercanos (*vecinos*).
3. **Clasificación:** devolver la etiqueta mayoritaria entre los k vecinos.
   Regresión: devolver la media aritmética de sus valores.

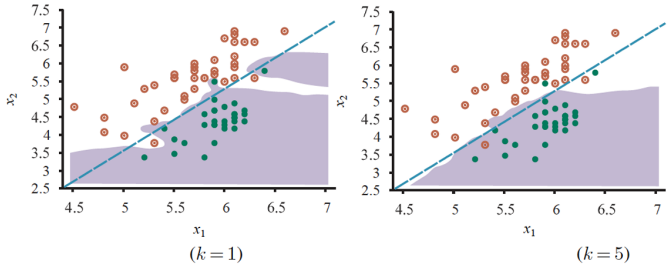


Figura 7: Fronteras de decisión de kNN con $k = 1$ (izq.) y $k = 5$ (der.). Un k mayor suaviza la frontera y reduce el sobreajuste. (Fuente: Diapositiva 37 del temario)

6.3. Hiperparámetro k y su Impacto

- $k = 1$: frontera de decisión muy irregular, alta varianza (*overfitting*).
- k grande: frontera más suave, mayor sesgo (*underfitting*).
- **Nota:** k es un hiperparámetro que debe ajustarse mediante validación cruzada.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

7. Redes Neuronales Artificiales (RNA)

7.1. Definición y Modelo de Neurona

> Una **Red Neuronal Artificial (RNA)** es un paradigma de procesamiento de información inspirado en la estructura del cerebro, compuesto por unidades interconectadas (neuronas) con pesos asociados.

Modelo matemático de neurona (McCulloch & Pitts, 1943):

$$y = \phi\left(\sum_{i=1}^{m} w_i \cdot x_i\right)$$

donde ϕ es la **función de activación** (escalón, sigmoidal, ReLU, gaussiana, etc.) y w_i son los pesos sinápticos.

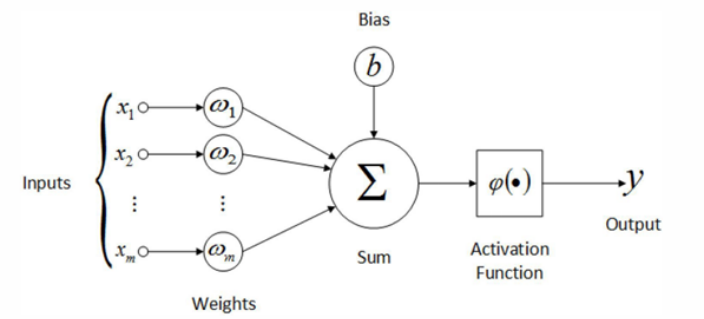


Figura 8: Modelo de neurona artificial con entradas x_i, pesos ω_i, bias b, función de activación φ y salida y. (Fuente: Diapositiva 40 del temario)

7.2. Arquitectura

La arquitectura de una RNA queda definida por:

- **Número de capas** (entrada, ocultas, salida).
- **Número de neuronas por capa.**
- **Tipo de conexiones** (densas, recurrentes, convolucionales...).
- **Perceptrón:** la RNA más sencilla; solo capa de entrada y de salida.
- **Perceptrón Multicapa (MLP):** incluye capas ocultas intermedias; puede aproximar cualquier función continua con suficiente precisión.

Una RNA de L capas puede verse como función anidada:

$$y = f_{\text{RNA}}(\mathbf{x}) = f_L\Big(f_{L-1}(\cdots f_1(\mathbf{x})\cdots)\Big)$$

7.3. Entrenamiento: Backpropagation

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Elimina la publicidad de este documento con 1 coin

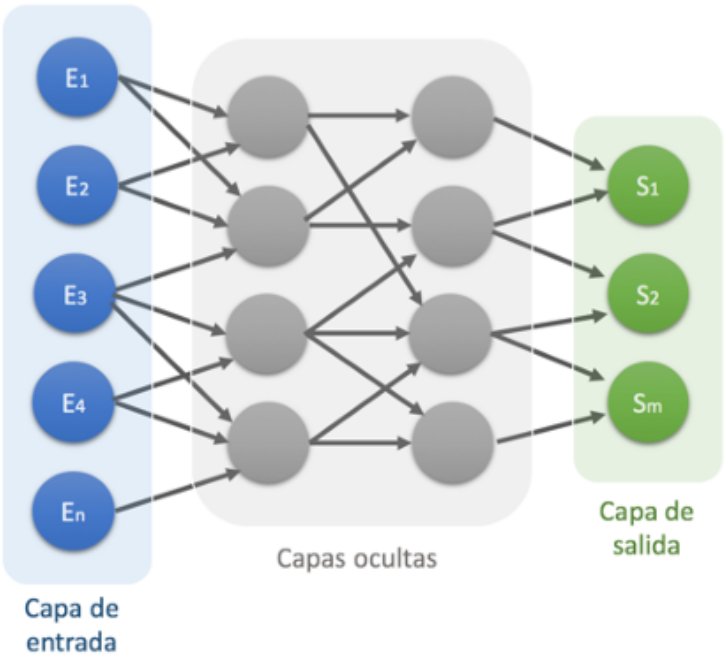


Figura 9: Arquitectura de una RNA con capa de entrada, capas ocultas y capa de salida. (Fuente: Diapositiva 42 del temario)

El algoritmo de **backpropagation** es el método estándar para aprender los pesos de una RNA. Es un proceso iterativo con dos etapas por iteración:

1. **Propagación hacia delante (*forward pass*):** se calculan las predicciones de la red con los pesos actuales y se obtiene el error.
2. **Propagación hacia atrás (*backward pass*):** se calcula el gradiente del error respecto a cada peso (regla de la cadena) y se actualizan los pesos.

- Los **pesos sinápticos** representan el conocimiento almacenado.
- Hiperparámetros definidos por el experto: número de capas, número de neuronas, función de activación, tasa de aprendizaje, etc.

7.4. Tipos de RNA

- Redes de función de base radial (RBF).
- Redes de Hopfield (asociativas / memoria).
- Redes de Kohonen (mapas autoorganizativos, aprendizaje no supervisado).
- Redes recurrentes (RNN, LSTM).

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Elimina la publicidad de este documento con 1 coin

Importante
Puedo eliminar la publi de este documento con 1 coin
¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

8. Aprendizaje Profundo (*Deep Learning*)

8.1. Definición y Motivación

El **aprendizaje profundo (DL)** hace referencia al entrenamiento de RNA con **múltiples capas ocultas** (> 2), que aprenden representaciones jerárquicas de los datos.

¿Por qué es necesario?

- Datos de alta dimensionalidad (imágenes $100 \times 100 \rightarrow 10\,000$ características).
- La cantidad de parámetros crece rápidamente: añadir una capa con s_l neuronas agrega $(s_{l-1} + 1) \cdot s_l$ parámetros.
- Las RNA poco profundas no capturan relaciones complejas en datos brutos.

8.2. Funcionamiento Jerárquico

- **Capa 1:** aprende características simples (bordes, frecuencias básicas).
- **Capa 2:** combina características simples en patrones más complejos.
- **Capa n:** aprende representaciones de alto nivel (caras, objetos, conceptos).

8.3. Reto: *Vanishing Gradient*

Las primeras capas de redes profundas se entrenan muy lentamente porque el gradiente se desvanece al propagarse hacia atrás (***vanishing gradient problem***). Soluciones: funciones de activación **ReLU**, conexiones residuales (*skip connections*), normalización de lotes (*batch normalization*).

8.4. Principales Arquitecturas DL

8.4.1. Autoencoders

Red cuya salida es idéntica a la entrada. Aprende una representación comprimida (*encoding*) útil para reducción de dimensionalidad y detección de anomalías.

8.4.2. Redes Convolucionales (CNN)

Especialmente adecuadas para datos con estructura espacial (imágenes, señales). Habitualmente tienen tres tipos de capas:

- **Convolucional:** aplica filtros aprendibles para detectar características locales.
- **Pooling:** reduce la dimensionalidad preservando las características relevantes.
- **Clasificadora (fully connected):** produce la predicción final.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

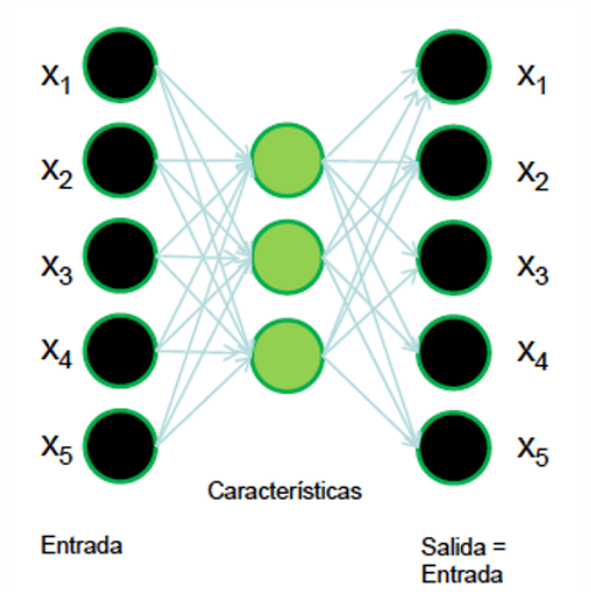


Figura 10: Arquitectura de un *autoencoder*: la capa central fuerza una representación compacta de la entrada. (Fuente: Diapositiva 52 del temario)

8.4.3. Redes Recurrentes (RNN / LSTM)

Incorporan **bucles de realimentación** que permiten persistir información a lo largo del tiempo. Adecuadas para secuencias: texto, audio, series temporales.

8.4.4. Otras Arquitecturas

GANs (*Generative Adversarial Networks*), *Capsule Networks*, redes siamesas, etc.

8.5. ¿Por Qué Tiene Tanto Éxito el DL Actualmente?

- **Datos masivos** disponibles (Big Data).
- **Computación paralela** en GPU.
- **Funciones de activación** ReLU y variantes, amigables para *backpropagation*.
- **Optimizadores** más robustos (Adam, RMSProp).
- **Plataformas SW**: TensorFlow, PyTorch, Keras.
- **Aprendizaje por transferencia (*Transfer Learning*):** reutilizar un modelo pre-entrenado y ajustar solo la última capa para una nueva tarea.

9. Resumen Comparativo de Técnicas

WUOLAH

Elimina la publicidad de este documento con 1 coin

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

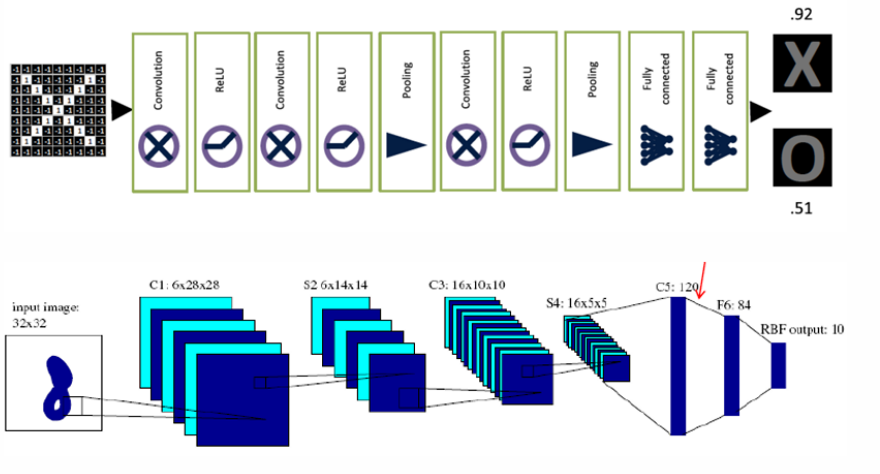


Figura 11: Arquitectura típica de una CNN para reconocimiento de dígitos. (Fuente: Diapositiva 53 del temario)

Técnica	Tipo	Fortalezas	Limitaciones
Árbol de decisión	Modelo	Interpretable, rápido	Sobreajuste si es profundo
SVM	Modelo	Máximo margen, robusto	Costoso con muchos datos
kNN	Instancias	Simple, sin entrenamiento	Lento en predicción, sensible a escala
RNA (MLP)	Modelo	Muy expresivo	Caja negra, necesita muchos datos
Deep Learning	Modelo	Estado del arte en imágenes/texto	Gran coste computacional

10. Puntos Clave para el Examen

1. **Tipos de aprendizaje:** supervisado, no supervisado, semi-supervisado, por refuerzo. Saber distinguirlos con ejemplos.
2. **Parámetros vs. hiperparámetros:** los parámetros los aprende el algoritmo; los hiperparámetros los fija el experto *antes* del entrenamiento.
3. **Navaja de Ockham:** ante hipótesis igualmente consistentes, preferir la más sencilla para evitar sobreajuste.
4. **Sobreajuste:** rendimiento en entrenamiento $\gg$ rendimiento en test. Indica falta de generalización.

WUOLAH
Elimina la publicidad de este documento con 1 coin
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Importante
Puedo eliminar la publi de este documento con 1 coin
¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

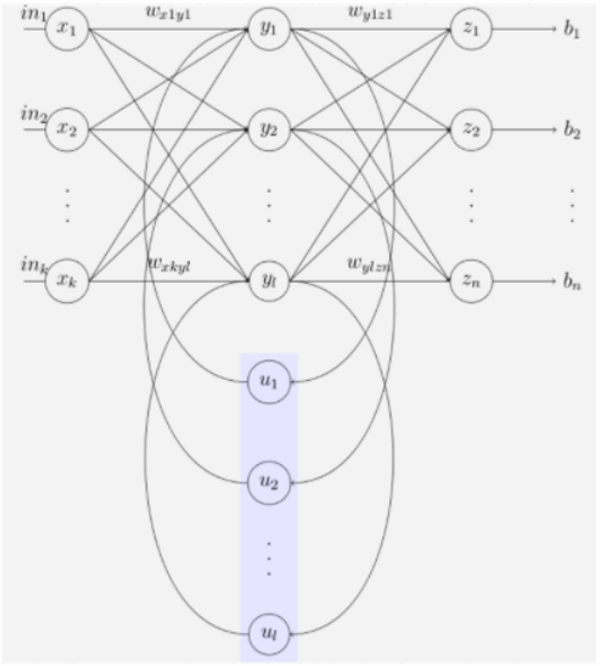


Figura 12: Arquitectura de una red recurrente con conexiones de realimentación entre neuronas ocultas. (Fuente: Diapositiva 54 del temario)

5. **Árbol de decisión:** estrategia divide y vencerás + ganancia de información. Función `Learn-Decision-Tree`.
6. **SVM:** maximiza el margen entre clases; los vectores soporte son los ejemplos más cercanos al hiperplano separador.
7. **kNN:** basado en instancias, no paramétrico. k es un hiperparámetro. Mayor $k \Rightarrow$ frontera más suave.
8. **RNA:** la salida de una neurona es $y = \phi(\sum w_i x_i)$. Se entrena con *backpropagation* (forward + backward pass).
9. **Deep Learning:** RNA con muchas capas; aprende representaciones jerárquicas. Problema del *vanishing gradient*. Principales arquitecturas: CNN (imágenes), RNN/LSTM (secuencias), Autoencoders (representación comprimida).
10. **Transfer Learning:** reutilizar red preentrenada y ajustar solo la última capa para una tarea nueva con pocos datos.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Bibliografía

1. Goodfellow, Bengio, Courville. *Deep Learning*. MIT Press, 2016.
2. Burkow, A. *The Hundred-Page Machine Learning Book*, 2019.
3. Mitchell, T.M. *Machine Learning*. McGraw-Hill, 1997.
4. Russell, S.; Norvig, P. *Artificial Intelligence: A Modern Approach*, 3.ª ed. Prentice Hall, 2010.
5. Rich, E.; Knight, K. *Inteligencia Artificial*. McGraw-Hill, 2006.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Elimina la publicidad de este documento con 1 coin